

Examen Parcial 3 Visión Computacional

Alumno:	Maldonado Acevedo Diego Rolando
Profesor:	Dr. Puente Montejano César Augusto
Semestre	2025-2026/I

DATASET MS COCO

MS COCO (Common Objects in Context) es uno de los datasets más importantes y ampliamente utilizados en el campo de la visión por computadora.

Creadores y Fecha de Creación

- **Creadores:** Microsoft Research
- **Año de lanzamiento inicial:** 2014
- **Equipo principal:** Tsung-Yi Lin, Genevieve Patterson, Matteo R. Ronchi, Yin Cui, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, Larry Zitnick, Piotr Dollár.

Contiene imágenes de objetos comunes en sus contextos naturales, con anotaciones detalladas que incluyen:

- Segmentación a nivel de píxel
- Cajas delimitadoras (bounding boxes)
- Etiquetas de categorías
- Keypoints para detección de poses humanas
- Leyendas (captions) para descripción de imágenes

Estadísticas Clave

- Número de imágenes: 330,000+
- Número de instancias de objetos: 2.5 millones
- Número de clases: 80 categorías de objetos
- Imágenes por división:
 - Entrenamiento: 118,287 imágenes
 - Validación: 5,000 imágenes
 - Test: 40,670 imágenes

Aplicaciones y Usos

MS COCO se ha utilizado en múltiples problemas de visión por computadora:

- Detección de objetos
- Segmentación de instancias
- Segmentación panóptica
- Detección de keypoints humanos
- Generación de descripciones de imágenes

- Segmentación de stuff (cosas)

Enlace de descarga: <https://cocodataset.org/#download>

Dataset Populares para la detección de objetos:

1. Pascal VOC (Visual Object Classes)

Características:

- 20 clases de objetos
- Aproximadamente 11,000 imágenes
- Incluye detección, segmentación y clasificación
- Fue el benchmark estándar antes de COCO

2. Open Images Dataset

Características:

- 9 millones de imágenes
- 600 clases de objetos
- 15.4 millones de cajas delimitadoras
- Creado por Google
- Incluye anotaciones de relaciones visuales y segmentación

3. ImageNet

Características:

- 14+ millones de imágenes
- 20,000 categorías (synsets)
- Incluye el subset ImageNet Large Scale Visual Recognition Challenge (ILSVRC)
- Ampliamente utilizado para clasificación y localización de objetos

Imágenes utilizadas para la prueba del programa de detección de objetos

Shibuya: <https://pin.it/6uKQjG6VR>

Street: <https://pin.it/1WKLhO963>

Pyd: <https://pin.it/4ppfH9lQi>

Video Shibuya: <https://pixabay.com/videos/shibuya-tokyo-japan-city-183046/>

a. Planteamiento del problema a resolver.

El problema consiste en desarrollar un sistema de detección de objetos que pueda identificar múltiples objetos en imágenes y videos utilizando el modelo YOLOv3 (You Only Look Once version 3). La solución es capaz de procesar tanto imágenes estáticas como flujos de video, dibujar bounding boxes alrededor de los objetos detectados y mostrar las clases correspondientes con sus niveles de confianza.

b. Descripción de la solución.

Flujo del Programa

1. **Inicialización:** Carga el modelo YOLOv3 y las clases COCO
2. **Preprocesamiento:** Convierte la imagen/video a blob
3. **Inferencia:** Pasa el blob a través de la red neuronal
4. **Postprocesamiento:** Filtra detecciones y aplica NMS
5. **Visualización:** Dibuja resultados y muestra/salva la salida

Modelo utilizado

Nombre completo: YOLOv3: An Incremental Improvement

Propuesto por: Joseph Redmon y Ali Farhadi (Universidad de Washington)

Año: 2018

Arquitectura:

Darknet-53 como backbone:

- 53 capas convolucionales
- Conexiones residuales (similar a ResNet)
- Sin capas fully connected
- Detección en 3 escalas diferentes (13x13, 26x26, 52x52) para detectar objetos pequeños, medianos y grandes
- Bounding box predictions: 3 cajas por celda en cada escala
- Feature Pyramid Network (FPN) para combinar características de diferentes niveles

Mejoras sobre YOLOv2:

- Multi-escala para mejor detección de objetos pequeños
- Más capas convolucionales
- Mejor clasificación con predicciones logísticas independientes
- 80 clases de MS COCO

Problema que resuelve:

Detección de objetos en tiempo real con balance entre velocidad y precisión:

- Identifica qué objetos hay en una imagen/video
- Localiza dónde están (bounding boxes)
- En una sola pasada de la red (a diferencia de métodos como R-CNN que requieren múltiples pasadas)

- Velocidad: ~30 FPS en GPU para aplicaciones en tiempo real (vigilancia, vehículos autónomos, robótica)

Ventaja principal: Procesa la imagen completa de una vez, no por regiones, lo que lo hace muy rápido comparado con métodos anteriores como Faster R-CNN.

```
# Cargar el modelo YOLOv3 entrenado con MS COCO
net = cv2.dnn.readNet('yolov3.weights', 'yolov3.cfg')
```

Carga el modelo YOLOv3 pre-entrenado desde los archivos de pesos (.weights) y configuración (.cfg). Los parámetros son:

- 'yolov3.weights': Archivo con los pesos entrenados del modelo
- 'yolov3.cfg': Archivo de configuración que define la arquitectura de la red

```
# Crear blob desde el frame
blob = cv2.dnn.blobFromImage(frame, 1/255.0, (416, 416), swapRB=True, crop=False)
```

Preprocesa la imagen para la red neuronal. Parámetros:

- frame: Imagen de entrada
- 1/255.0: Factor de escala para normalizar los valores de píxeles (0-255 → 0-1)
- (416, 416): Tamaño al que se redimensiona la imagen (tamaño de entrada de YOLOv3)
- swapRB=True: Intercambia los canales Red y Blue (OpenCV usa BGR, YOLO espera RGB)
- crop=False: No recortar la imagen, solo redimensionar

```
# Pasar el blob a través de la red
net.setInput(blob)
outputs = net.forward(output_layers)
```

- setInput(): Establece el blob como entrada de la red
- forward(): Realiza la inferencia y obtiene las salidas de las capas especificadas

```
# Aplicar Non-Maximum Suppression para eliminar detecciones duplicadas
indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)
```

Aplica Non-Maximum Suppression para eliminar detecciones duplicadas. Parámetros:

- boxes: Lista de bounding boxes detectados
- confidences: Lista de niveles de confianza
- 0.5: Umbral de confianza mínimo
- 0.4: Umbral de NMS para suprimir boxes que se superponen demasiado

```
# Dibujar rectángulo y etiqueta
cv2.rectangle(frame, (x, y), (x + w, y + h), color, 2)
text = f"{label}: {confidence:.2f}"
cv2.putText(frame, text, (x, y - 5), font, 0.5, color, 2)
```

Dibujan los bounding boxes y etiquetas en la imagen.

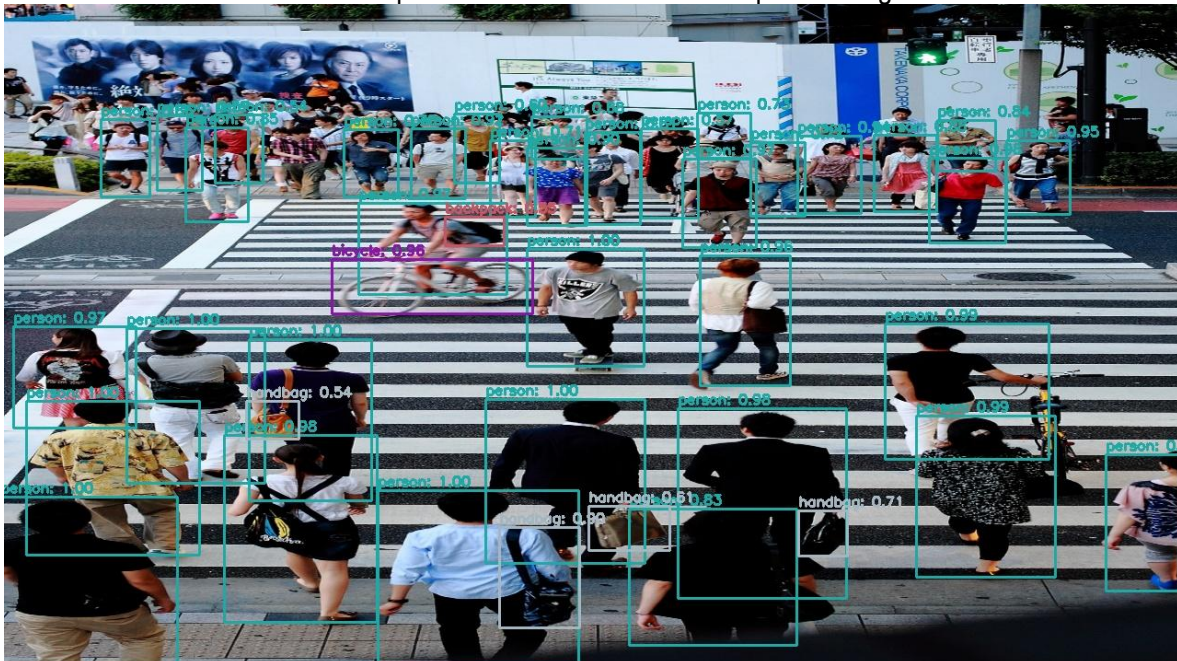
c. Descripción de los resultados.

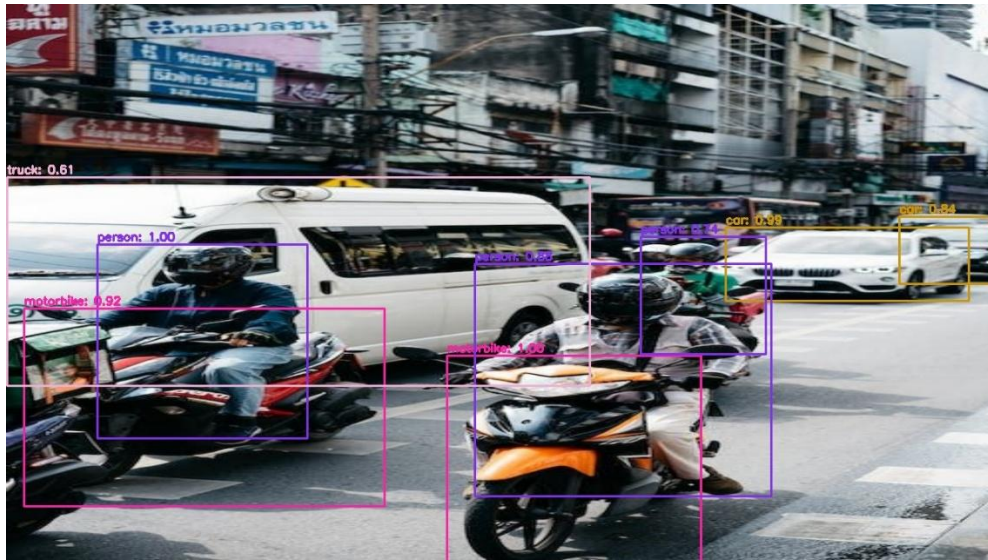


Primero se seleccionaron las imágenes desde Pinterest para tener un numero mayor de objetos dentro de una imagen.



También un video llamado shib.mp4 el cual está dentro de la carpeta de img.





Posteriormente al hacer el procesamiento de estas imágenes obtuve los siguientes resultados. El resultado del video también se encuentra en la carpeta de img, este se hizo el procesamiento con el cpu por lo que fue lento el procesamiento de este, se podría implementar que fuese procesado con la gpu.

d. Discusión.

La implementación de YOLOv3 para detección de objetos ha representado una valiosa experiencia de aprendizaje en múltiples dimensiones. Este trabajo no solo me permitió aplicar conceptos teóricos de visión por computadora y redes neuronales profundas, sino que también me confrontó con los desafíos prácticos del desarrollo de software en inteligencia artificial.

La verdad, esta materia de visión computacional ha sido de las más interesantes que he llevado en toda la carrera.